

Beginner Level Questions

1. Print a simple message in Python.

Approach: Use print().

2. Check whether a number is even or odd.

Approach: Use the modulo operator: $n \% 2 == 0$.

3. Concatenate two strings.

Approach: Use + or an f-string.

4. Find the maximum of three numbers.

Approach: Use max(a, b, c).

5. Count vowels in a string.

Approach: Iterate through the string and test membership in aeiou.

6. Calculate the factorial of a number.

Approach: Use iteration or recursion; define $0! = 1$.

7. Convert a numeric string to an integer.

Approach: Use int().

8. Calculate the area of a rectangle.

Approach: Area = length $\times$ width.

9. Calculate factorial using a reusable function.

Approach: Create factorial(n) with a base case and recursive/iterative multiplication.

10. Check whether a number is prime.

Approach: Test divisibility up to $\sqrt{n}$.

11. Check whether a number is a palindrome.

Approach: Compare the value/string with its reverse.

12. Check whether a number is an Armstrong number.

Approach: Raise each digit to the number of digits and compare the sum with the original.

13. Check whether a number is perfect.

Approach: Sum proper divisors and compare the sum with the number.

14. Check whether a year is a leap year.

Approach: Divisible by 400, or divisible by 4 but not by 100.

15. Calculate factorial again as a standard recursion exercise.

Approach: Use a clear base case $n == 0$ and recursive multiplication.

16. Generate the first n Fibonacci terms.

Approach: Maintain the previous two terms and append their sum.

17. Calculate the sum of digits of an integer.

Approach: Repeatedly take $n \% 10$ and reduce n using integer division.

18. Convert a decimal number to binary.

Approach: Use bin(n)[2:] or implement repeated division by 2.

19. Check whether a number is a prime factor of another number.

Approach: The candidate must be prime and divide the target exactly.

20. Find all prime numbers in a given range.

Approach: Test each candidate with a prime-checking helper.

Intermediate Level Questions

21. Merge two dictionaries.

Approach: Use dictionary unpacking, update(), or the | operator in modern Python.

22. Find common elements in two lists.

Approach: Use set intersection when order/duplicates are not required.

23. Remove duplicates from a list.

Approach: Use a set for uniqueness; preserve order with a seen set if required.

24. Check whether a string is a palindrome.

Approach: Compare s with s[::-1].

25. Find the longest word in a sentence.

Approach: Split into words and select the item with maximum length.

26. Find the first non-repeating character.

Approach: Count characters first, then scan the original string.

27. Count uppercase letters in a string.

Approach: Use char.isupper() while iterating.

28. Reverse a string.

Approach: Use slicing s[::-1] or a loop.

29. Convert a list of characters into a string.

Approach: Use "".join(chars).

30. Split a sentence into words.

Approach: Use sentence.split().

31. Join a list of strings.

Approach: Use separator.join(items).

32. Count occurrences of a character.

Approach: Use str.count() or a frequency dictionary.

33. Count digits, letters, and spaces in a string.

Approach: Use isdigit(), isalpha(), and isspace().

34. Find the largest element in a list.

Approach: Use max().

35. Find the smallest element in a list.

Approach: Use min().

36. Merge two sorted lists.

Approach: For a simple solution, concatenate and sort; for optimal merging, use two pointers.

37. Explain and use a Python list.

Approach: Lists are mutable ordered collections; use append(), remove(), indexing, and slicing.

38. Explain and use a Python dictionary.

Approach: Dictionaries map keys to values and support fast key-based lookup.

39. Explain and use a set.

Approach: Sets store unique hashable elements and support union/intersection operations.

40. Merge two sorted lists using Python tools.

Approach: `sorted(a + b)` is concise; two-pointer merging is more algorithmic.

41. Find the longest consecutive sequence in an unsorted list.

Approach: Put values in a set and start sequences only at values with no predecessor.

42. Solve the Two Sum problem.

Approach: Store previously seen values in a dictionary and look for `target - current`.

43. Group anagrams.

Approach: Use a normalized key such as sorted characters or a character-count tuple.

44. Traverse a binary tree level by level.

Approach: Use breadth-first search with `collections.deque`.

45. Explain memoization and use it in an algorithm.

Approach: Cache previously computed results to avoid repeated work.

46. Explain nested loops and their uses.

Approach: A loop inside another loop is useful for matrices and pairwise comparisons; analyze time complexity carefully.

47. Explain Pandas Series and DataFrame.

Approach: Series is one-dimensional labelled data; DataFrame is two-dimensional tabular data.

48. Perform sorting and searching in Python.

Approach: Use `sorted()/sort()` for ordering and membership/index methods for basic searches.

49. Represent a graph in Python.

Approach: An adjacency list can be represented with a dictionary mapping vertices to neighbors.

50. Explain vectorized operations.

Approach: Apply operations to whole NumPy arrays instead of Python loops for efficient numerical work.

51. Explain Python profiling tools.

Approach: Tools such as `cProfile` and `timeit` help locate performance bottlenecks.

52. Explain data enrichment.

Approach: Combine existing data with relevant external or reference information, often through joins/merges.

Advanced Level Questions

53. Implement binary search.

Approach: On sorted data, repeatedly halve the search interval; $O(\log n)$ time.

54. Flatten a nested list.

Approach: Use recursion to process nested lists or an explicit stack for deep structures.

55. Check whether a binary tree is balanced.

Approach: Compare subtree heights recursively; return failure early for efficiency.

56. Find the maximum subarray sum.

Approach: Use Kadane's algorithm with running and global sums; $O(n)$.

57. Find the intersection of two sets.

Approach: Use set1 & set2.

58. Implement a simple calculator.

Approach: Dispatch on the requested operation and handle division by zero.

59. Check whether a number is a perfect square.

Approach: Compare the square of the integer square root with the original number.

60. Find the GCD of two numbers.

Approach: Use the Euclidean algorithm.

61. Convert temperatures from Celsius to Fahrenheit.

Approach: $F = C \times 9/5 + 32$.

62. Implement a queue using collections.deque.

Approach: enqueue with append(); dequeue with popleft().

63. Explain a Python metaclass.

Approach: A metaclass controls class creation; type is Python's default metaclass.

64. Differentiate @staticmethod and @classmethod.

Approach: staticmethod receives neither instance nor class automatically; classmethod receives cls.

65. Explain the with statement.

Approach: It uses a context manager to acquire/release resources reliably, even on errors.

66. Explain magic (dunder) methods.

Approach: Special methods such as `__init__`, `__str__`, `__len__`, and `__add__` customize object behavior.

67. Implement a singleton pattern.

Approach: Control instance creation so that repeated construction returns the same object.

68. Explain multithreading and the CPython GIL.

Approach: Threads are useful for I/O-bound work; the GIL limits parallel Python bytecode execution in standard CPython.

69. Explain coroutines and asynchronous programming.

Approach: Use `async def` and `await` for cooperative asynchronous execution.

70. Explain __slots__.

Approach: Restrict instance attributes and can reduce per-instance memory overhead.

71. Use Pandas for data manipulation.

Approach: Create a DataFrame and transform columns with vectorized operations.

72. Read a CSV file using Pandas.

Approach: Use `pd.read_csv()` and inspect the resulting DataFrame.

73. Explain Selenium and give a basic use case.

Approach: Selenium automates browser interaction for testing and web workflows.

74. Handle missing values in a Pandas DataFrame.

Approach: Use `dropna()` to remove missing values or `fillna()` to impute them.

75. Explain NumPy and its importance.

Approach: NumPy provides efficient multidimensional arrays and numerical operations.

76. Use Pytest or Unittest for testing.

Approach: Write test functions/cases with assertions and run them through the chosen framework.

77. Explain TensorFlow or PyTorch.

Approach: They are major frameworks for machine learning and deep learning.

78. Automate Excel tasks with Python.

Approach: Use openpyxl for reading/writing .xlsx workbooks.

79. Explain Flask or Dash.

Approach: Flask supports web applications; Dash is oriented toward analytical/data-visualization applications.

80. Improve Python code style and quality.

Approach: Use PEP 8, formatters such as Black, and linters such as Pylint/Ruff.

Core Python Concepts and Syntax

1. Differentiate a list and a tuple.

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

2. Explain Python memory management.

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

3. Explain exception handling with try/except.

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

4. Explain conditional statements: if, elif, else.

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

5. How is a function defined in Python?

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

6. What is PEP 8 and why does it matter?

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

7. How does indentation define Python blocks?

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

8. What is a class and how is an object created?

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

9. What are Python's major built-in data types?

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

10. What is package management and how is pip used?

Revision point: Be ready to explain the concept, show a short example, and mention a practical use.

Scenario-Based and Professional Questions

1. How would you perform API interactions in Python?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

2. How would you handle I/O-bound tasks efficiently?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

3. How can assertions be used for testing?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

4. How can tests be driven by external data?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

5. How should environment variables be used for secrets?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

6. How would you identify and fix performance bottlenecks?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

7. Which application performance metrics should be monitored?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

8. What secure coding practices should be followed?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

9. How should HTTP status codes be handled?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

10. How would you perform web scraping?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

11. How would you approach a real-world problem using Python?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

12. How should API keys and credentials be stored securely?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

13. How would you handle concurrent API requests?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

14. How can assertions validate business/data rules?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

15. How would you optimize a slow web-scraping workflow?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

16. How can retry logic make API/scraping code resilient to network failures?

Interview focus: Explain the design choice, error handling, security/performance considerations, and a small implementation outline.

How to Practice Python Coding Questions

1. Build fundamentals first: variables, data types, operators, conditions, loops, functions, and basic data structures.
2. Practice number and string problems such as primes, factorials, palindromes, Fibonacci, and frequency counting.
3. Become fluent with lists, tuples, dictionaries, sets, slicing, comprehensions, and common built-in functions.
4. Learn algorithmic patterns: two pointers, hashing, recursion, binary search, BFS, and dynamic programming basics.
5. During interviews, clarify the input/output, discuss edge cases, state the approach, then code.
6. Prefer readable code and explain time and space complexity for algorithmic problems.
7. Test normal cases, boundary cases, empty inputs, duplicates, negative values, and invalid inputs when relevant.
8. Use small practice sessions and timed mock problems to improve speed without sacrificing correctness.

Suggested Practice Order

Stage	Topics
1	Variables, input/output, operators, conditions, loops
2	Strings, numbers, lists, tuples, sets, dictionaries
3	Functions, recursion, comprehensions, exception handling
4	Searching, sorting, hashing, two pointers, queues, trees, graphs
5	Pandas, NumPy, testing, APIs, web scraping, performance
6	Timed interview problems + explanation of time/space complexity

Reference

CCBP. "Top 100+ Python Coding Questions for Interview." Published 17 June 2026.
<https://www.ccbp.in/blog/articles/python-coding-questions>